

LAB ASSIGNMENT - 11.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description -1 – (Stack Using AI Guidance)

- Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

Code:

#Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations. Expected Output: A Python Stack class supporting push, pop, peek, and empty - check operations with proper documentation.

```
class Stack:
```

```
    """
```

```
    A Stack data structure that supports basic stack operations: push, pop, peek, and empty check.
```

```
    """
```

```
    def __init__(self):
```

```
        """Initialize an empty stack."""
```

```
        self.stack = []
```

```
    def push(self, item):
```

```
        """
```

```
        Add an item to the top of the stack.
```

Parameters:

item: The item to be added to the stack.

```
"""
```

```
self.stack.append(item)
```

def pop(self):

```
"""
```

Remove and return the item at the top of the stack.

Returns:

The item at the top of the stack.

Raises:

IndexError: If the stack is empty.

```
"""
```

```
if self.empty():
```

```
    raise IndexError("Pop from an empty stack")
```

```
return self.stack.pop()
```

def peek(self):

```
"""
```

Return the item at the top of the stack without removing it.

Returns:

The item at the top of the stack.

Raises:

IndexError: If the stack is empty.

```
"""
```

```
if self.empty():
```

```
    raise IndexError("Peek from an empty stack")
```

```
return self.stack[-1]
```

```
def empty(self):
```

```
    """
```

```
    Check if the stack is empty.
```

```
    Returns:
```

```
    True if the stack is empty, False otherwise.
```

```
    """
```

```
    return len(self.stack) == 0
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    stack = Stack()
```

```
    stack.push(1)
```

```
    stack.push(2)
```

```
    stack.push(3)
```

```
    print(stack.peek()) # Output: 3
```

```
    print(stack.pop()) # Output: 3
```

```
    print(stack.peek()) # Output: 2
```

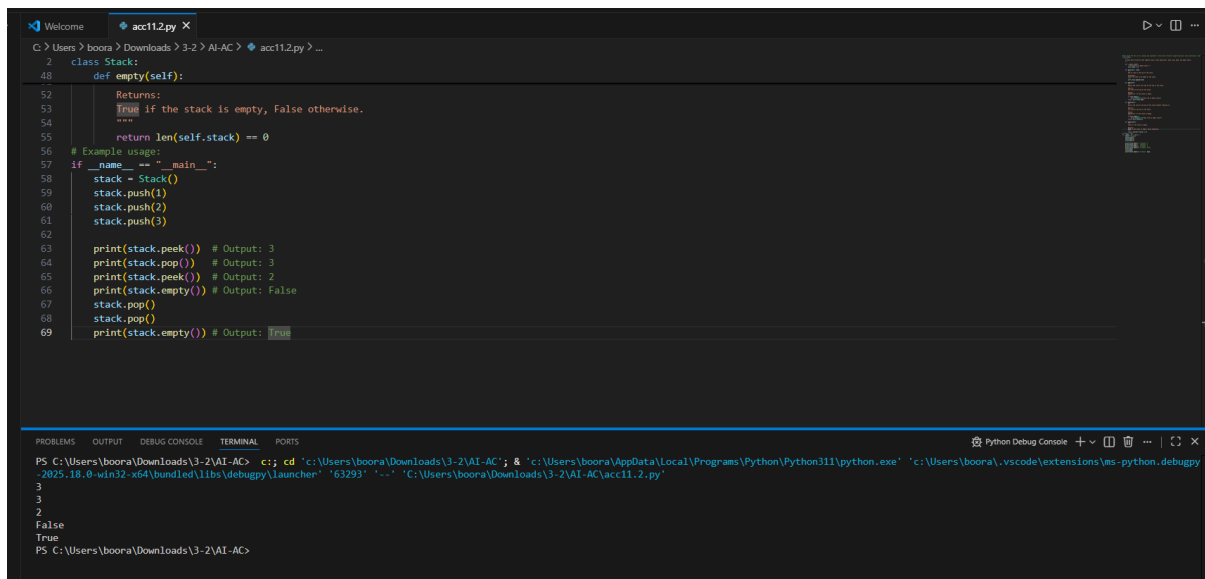
```
    print(stack.empty()) # Output: False
```

```
    stack.pop()
```

```
    stack.pop()
```

```
    print(stack.empty()) # Output: True
```

Output:



```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0

# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek()) # Output: 3
    print(stack.pop()) # Output: 3
    print(stack.peek()) # Output: 2
    print(stack.is_empty()) # Output: False
    stack.pop()
    print(stack.is_empty()) # Output: True
```

Python Debug Console

```
PS C:\Users\boora\Downloads\3-2\AI-AC> c:\cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63293' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\acc11.2.py'
3
3
2
False
True
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

The Stack data structure was implemented with the help of AI to understand *LIFO (Last In First Out)* behavior. AI guidance helped in designing a Python class with methods such as *push, pop, peek, and isEmpty*. This improved understanding of how elements are added and removed from the top of the stack and how stack operations are implemented efficiently.

Task Description -2 – (Queue Design)

- Task: Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

Code:

#Task: Use AI assistance to create a Queue data structure following FIFO principles

Expected Output: A complete Queue implementation including enqueue, dequeue, front element access, and size calculation.

class Queue:

"""

A Queue data structure that follows FIFO (First In, First Out) principles.

```
"""
```

```
def __init__(self):
```

```
    """Initialize an empty queue."""
```

```
    self.queue = []
```

```
def enqueue(self, item):
```

```
    """
```

```
    Add an item to the end of the queue.
```

```
    Parameters:
```

```
    item: The item to be added to the queue.
```

```
    """
```

```
    self.queue.append(item)
```

```
def dequeue(self):
```

```
    """
```

```
    Remove and return the item at the front of the queue.
```

```
    Returns:
```

```
    The item at the front of the queue.
```

```
    Raises:
```

```
    IndexError: If the queue is empty.
```

```
    """
```

```
    if self.empty():
```

```
        raise IndexError("Dequeue from an empty queue")
```

```
return self.queue.pop(0)
```

```
def front(self):
```

```
    """
```

Return the item at the front of the queue without removing it.

Returns:

The item at the front of the queue.

Raises:

IndexError: If the queue is empty.

```
    """
```

```
    if self.empty():
```

```
        raise IndexError("Front from an empty queue")
```

```
    return self.queue[0]
```

```
def size(self):
```

```
    """
```

Return the number of items in the queue.

Returns:

The size of the queue.

```
    """
```

```
    return len(self.queue)
```

```
def empty(self):
```

```
    """
```

Check if the queue is empty.

Returns:

True if the queue is empty, False otherwise.

```
"""
```

```
return len(self.queue) == 0
```

Example usage:

```
if __name__ == "__main__":
```

```
    queue = Queue()
```

```
    queue.enqueue(1)
```

```
    queue.enqueue(2)
```

```
    queue.enqueue(3)
```

```
    print(queue.front()) # Output: 1
```

```
    print(queue.dequeue()) # Output: 1
```

```
    print(queue.front()) # Output: 2
```

```
    print(queue.size()) # Output: 2
```

```
    queue.dequeue()
```

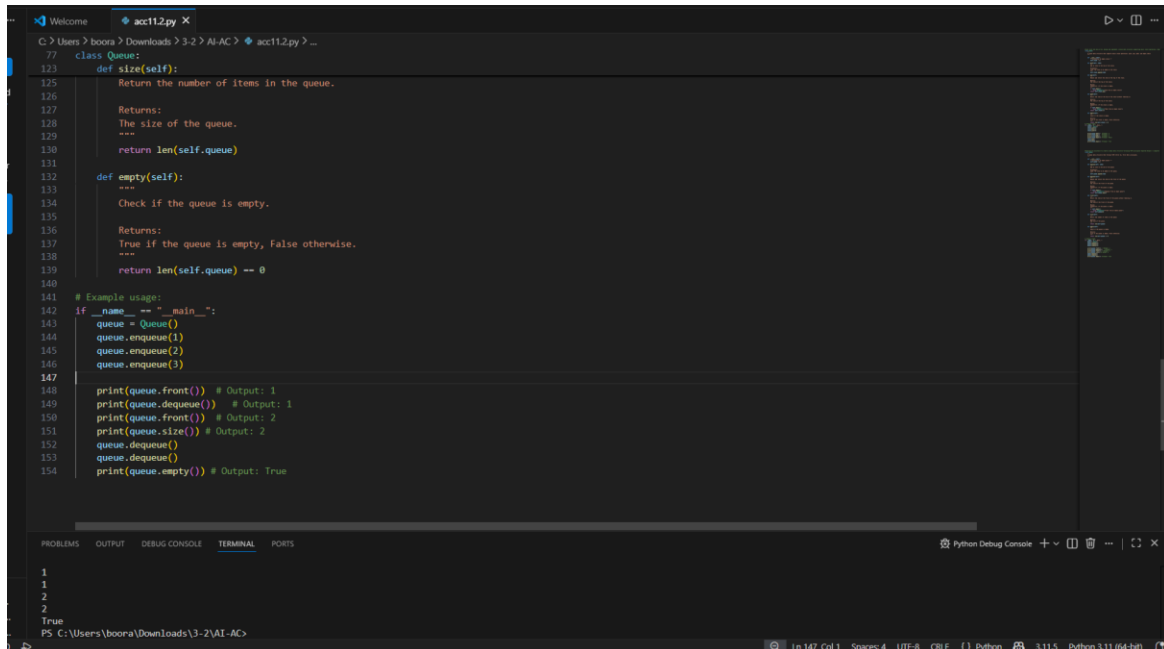
```
    queue.dequeue()
```

```
    print(queue.empty()) # Output: True
```

Justification:

AI assistance was used to design a Queue following the *FIFO (First In First Out)* principle. The implementation includes operations like *enqueue, dequeue, front, and size*. This task helped in understanding how queues manage data sequentially and how they are used in real-life applications such as task scheduling and buffering.

Output:



```
77 class Queue:
78     def size(self):
79         """
80         Return the number of items in the queue.
81
82         Returns:
83             The size of the queue.
84         """
85         return len(self.queue)
86
87     def empty(self):
88         """
89         Check if the queue is empty.
90
91         Returns:
92             True if the queue is empty, False otherwise.
93         """
94         return len(self.queue) == 0
95
96 # Example usage:
97 if __name__ == "__main__":
98     queue = Queue()
99     queue.enqueue(1)
100     queue.enqueue(2)
101     queue.enqueue(3)
102
103     print(queue.front()) # Output: 1
104     print(queue.dequeue()) # Output: 1
105     print(queue.front()) # Output: 2
106     print(queue.size()) # Output: 2
107     queue.dequeue()
108     queue.dequeue()
109     print(queue.empty()) # Output: True
```

1
1
2
2
True
PS C:\Users\boora\Downloads\3-2\AI-AC>

Task Description -3 –(Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

Code:

Utilize AI to build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality

Node class

class Node:

```
def __init__(self, data):
```

```
    self.data = data
```



```
        self.next = None

# Linked List class
class SinglyLinkedList:

    def __init__(self):

        self.head = None


    # Insert node at end

    def insert(self, data):

        new_node = Node(data)

        if self.head is None:

            self.head = new_node

            return

        temp = self.head

        while temp.next:

            temp = temp.next

        temp.next = new_node

    # Traverse and display list

    def display(self):

        temp = self.head

        while temp:

            print(temp.data, end=" -> ")

            temp = temp.next

        print("None")


# Create linked list
```

```
ll = SinglyLinkedList()
```

```
# Inserting elements
```

```
ll.insert(10)
```

```
ll.insert(20)
```

```
ll.insert(30)
```

```
ll.insert(40)
```

```
# Display list
```

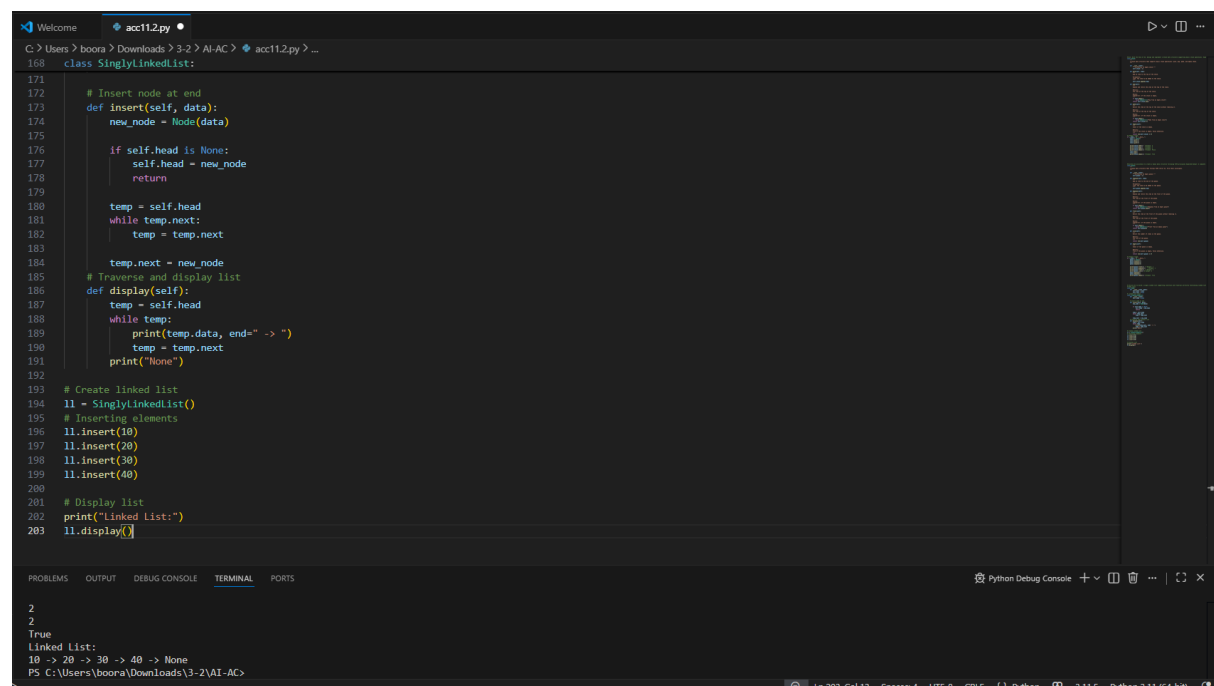
```
print("Linked List:")
```

```
ll.display()
```

Justification:

With the support of AI, a *Singly Linked List* was developed to understand dynamic data structures. The program includes *node creation, insertion, and traversal*. This helped in learning how linked lists store data in nodes connected through pointers and how they allow flexible memory usage compared to arrays.

Output:



```
class SinglyLinkedList:
    # Insert node at end
    def insert(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = new_node

    # Traverse and display list
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

# Create linked list
ll = SinglyLinkedList()
# Inserting elements
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.insert(40)

# Display list
print("Linked List:")
ll.display()
```

2
2
True
Linked List:
10 -> 20 -> 30 -> 40 -> None
PS C:\Users\boona\Downloads\3-2\AI-AC>

Task Description -4 – (Binary Search Tree Operations)

- Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

Code:

Utilize AI to build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality

Node class

class Node:

def __init__(self, data):

self.data = data

self.next = None

Linked List class

class SinglyLinkedList:

def __init__(self):

self.head = None

Insert node at end

def insert(self, data):

new_node = Node(data)

if self.head is None:

self.head = new_node

return

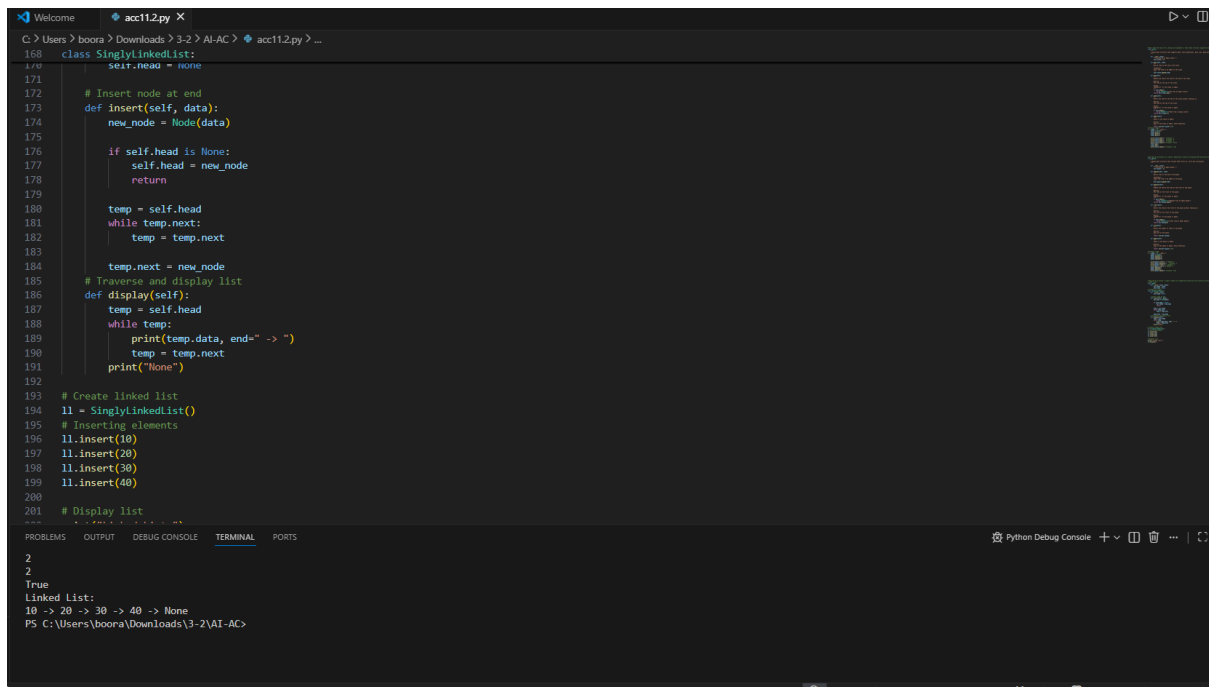
```
temp = self.head
while temp.next:
    temp = temp.next

temp.next = new_node
# Traverse and display list
def display(self):
    temp = self.head
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")

# Create linked list
ll = SinglyLinkedList()
# Inserting elements
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.insert(40)

# Display list
print("Linked List:")
ll.display()
```

Output:



```
168 class SinglyLinkedList:
169     self.head = None
170
171     # Insert node at end
172     def insert(self, data):
173         new_node = Node(data)
174
175         if self.head is None:
176             self.head = new_node
177             return
178
179         temp = self.head
180         while temp.next:
181             temp = temp.next
182
183         temp.next = new_node
184     # Traverse and display list
185     def display(self):
186         temp = self.head
187         while temp:
188             print(temp.data, end=" -> ")
189             temp = temp.next
190         print("None")
191
192     # Create linked list
193     ll = SinglyLinkedList()
194     # Inserting elements
195     ll.insert(10)
196     ll.insert(20)
197     ll.insert(30)
198     ll.insert(40)
199
200     # Display list
201
202 2
203 2
204 True
205 Linked List:
206 10 -> 20 -> 30 -> 40 -> None
207 PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

AI guidance was used to implement a *Binary Search Tree (BST)* focusing on *node insertion and in-order traversal*. This task helped in understanding how BST organizes data in a sorted structure and allows efficient searching, insertion, and traversal operations.

Task Description -5 – (Hash Table Implementation)

- Task: Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

Code:

#Task: Create a hash table using AI with collision handling Expected Output: Hash table supporting insert, search, and delete using chaining or open.

class HashTable:

```

def __init__(self, size=10):
    self.size = size
    self.table = [[] for _ in range(size)]

def _hash(self, key):
    return hash(key) % self.size

def insert(self, key, value):
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index][i] = (key, value) # Update existing key
            return
    self.table[index].append((key, value)) # Insert new key-value pair

def search(self, key):
    index = self._hash(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return None # Key not found

def delete(self, key):
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            del self.table[index][i] # Remove the key-value pair
            return True

```

```
    return False # Key not found
```

Example usage:

```
if __name__ == "__main__":
```

```
    hash_table = HashTable()
```

```
    hash_table.insert("name", "Alice")
```

```
    hash_table.insert("age", 30)
```

```
    hash_table.insert("city", "New York")
```

```
    print(hash_table.search("name")) # Output: Alice
```

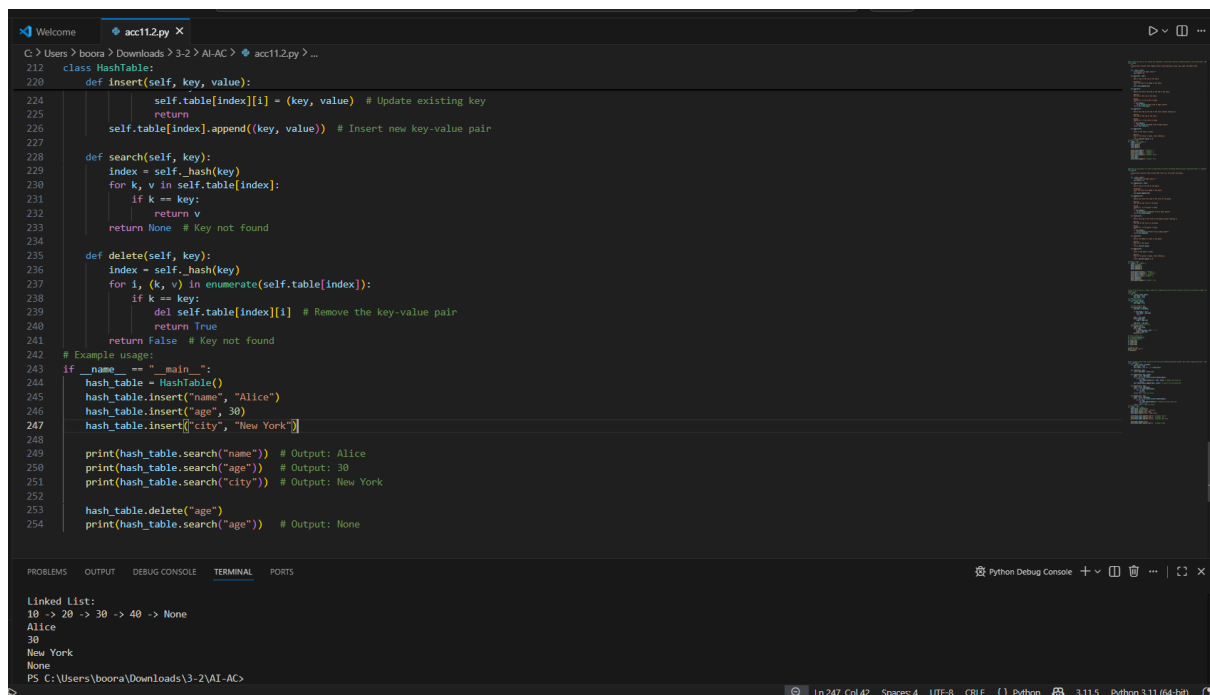
```
    print(hash_table.search("age")) # Output: 30
```

```
    print(hash_table.search("city")) # Output: New York
```

```
    hash_table.delete("age")
```

```
    print(hash_table.search("age")) # Output: None
```

Output:



```
212 class HashTable:
213     def __init__(self):
214         self.table = {}
215
216     def insert(self, key, value):
217         if key in self.table:
218             self.table[key].append((key, value)) # Update existing key
219         else:
220             self.table[key] = [(key, value)] # Insert new key-value pair
221
222     def search(self, key):
223         index = self._hash(key)
224         for k, v in self.table[index]:
225             if k == key:
226                 return v
227         return None # Key not found
228
229     def delete(self, key):
230         index = self._hash(key)
231         for i, (k, v) in enumerate(self.table[index]):
232             if k == key:
233                 del self.table[index][i] # Remove the key-value pair
234         return True
235
236     # Example usage:
237 if __name__ == "__main__":
238     hash_table = HashTable()
239     hash_table.insert("name", "Alice")
240     hash_table.insert("age", 30)
241     hash_table.insert("city", "New York")
242
243     print(hash_table.search("name")) # Output: Alice
244     print(hash_table.search("age")) # Output: 30
245     print(hash_table.search("city")) # Output: New York
246
247     hash_table.delete("age")
248     print(hash_table.search("age")) # Output: None
```

Linked List:
10 -> 20 -> 30 -> 40 -> None
Alice
30
New York
None

PS C:\Users\boora\Downloads\3-2\AI-AC>

Justification*

A *Hash Table* was implemented with AI support to understand fast data retrieval using *hashing techniques*. Collision handling was managed using **chaining, and the table supports **insert, search, and delete operations*. This task demonstrates how hash tables provide efficient data storage and quick access in many applications such as databases and caching systems.